

第七讲 共享存储编程

何克晶

kejinghe@gmail.com

华南理工大学

计算机科学与工程学院

共享存储编程

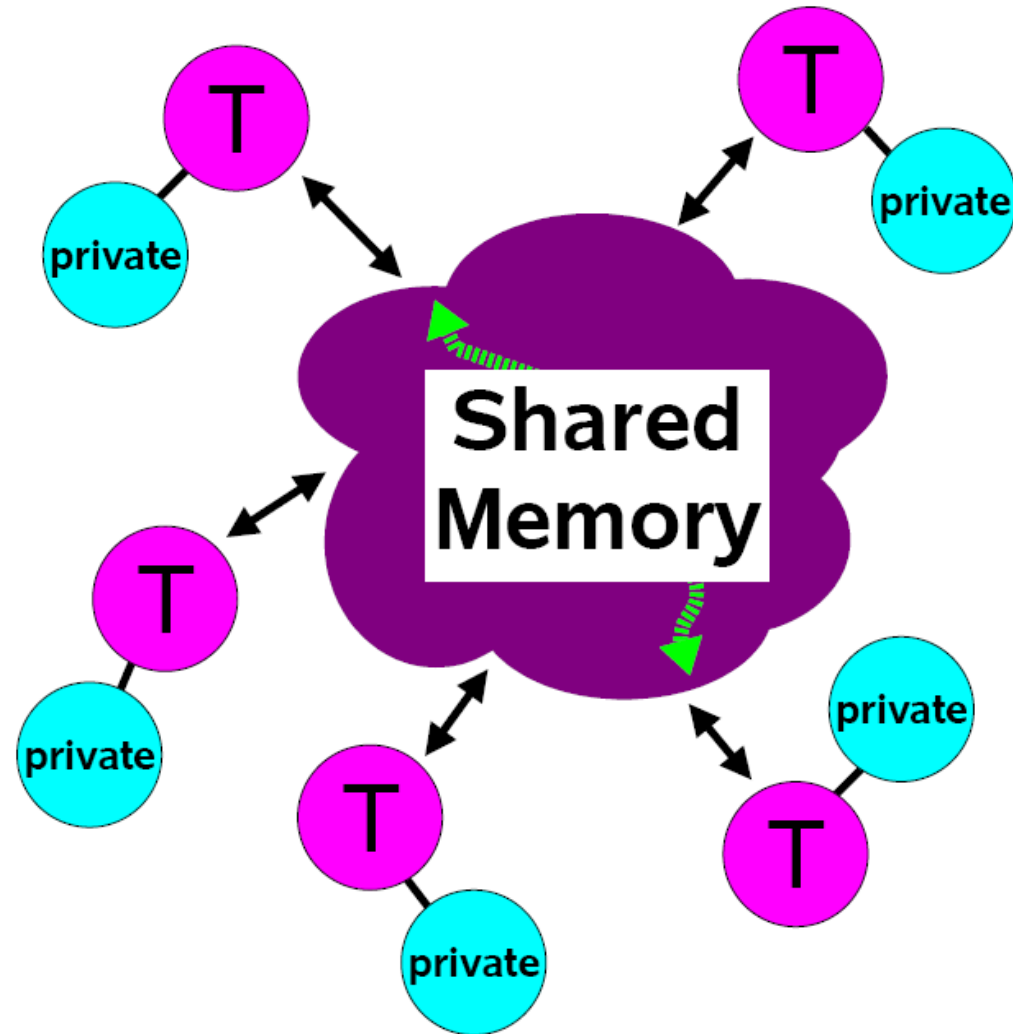
多线程

OpenMP Classic

OpenMP Offload

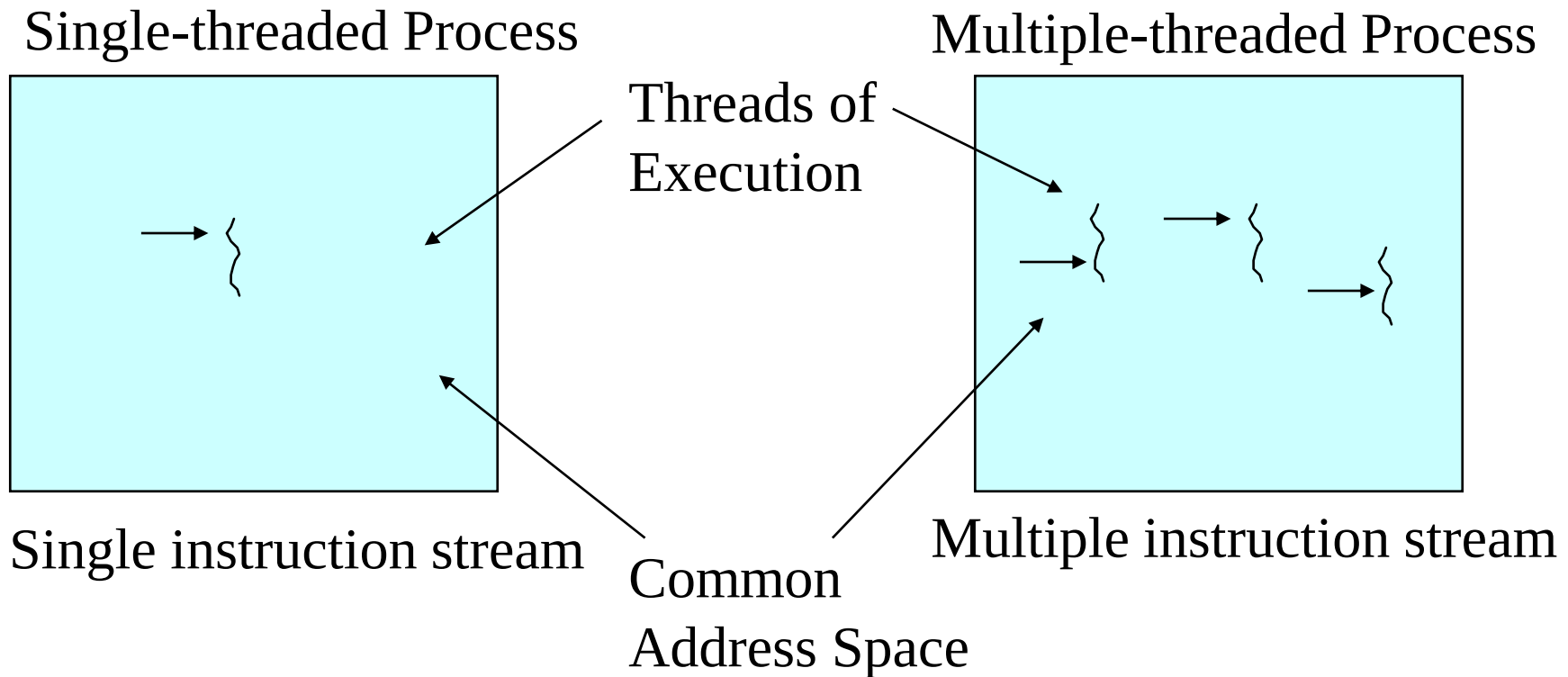
Shared Memory Model

- All threads have access to the same, globally shared, memory
- Data can be shared or private
- Shared data is accessible by all threads
- Private data can be accessed only by the threads that owns it
- Data transfer is transparent to the programmer
- Synchronization takes place, but it is mostly implicit



Single and Multithreaded Processes

Threads are light-weight processes within a process.



What are Threads?

- A piece of code that run in concurrent with other threads.
- Each thread is a statically ordered sequence of instructions.
- Threads are being extensively used express concurrency on both single and multiprocessors machines.
- Programming a task having multiple threads of control – Multithreading or Multithreaded Programming.

Java Threads

- Java has built in thread support for Multithreading
- Synchronization
- Thread Scheduling
- Inter-Thread Communication:
 - currentThread start setPriority
 - yield run getPriority
 - sleep stop

Java Garbage Collector is a low-priority thread.

Threading Mechanisms

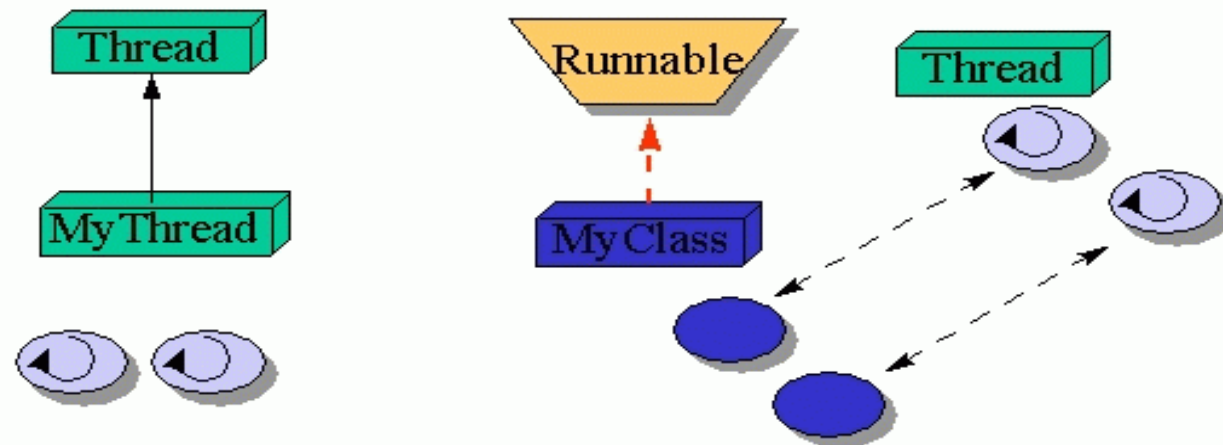
- Create a class that extends the Thread class

```
class MyThread extends Thread
```

- Create a class that implements the Runnable interface

```
class MyThread implements Runnable
```

Threading Mechanisms



Example: 3 Threads

```
class myth extends Thread
{
    myth (String name)
    {
        super(name);
    }
    public void run()
    {
        for(int i=1;i<=5;i++)
        {
            System.out.println("\t From "+getName()+" :
i= "+i);
        }
        System.out.println("Exit from "+getName());
    }
}

class ThreadTest
{
    public static void main(String args[])
    {
        new myth("ThreadA").start();
        new myth("ThreadB").start();
        new myth("ThreadC").start();
    }
}
```

```
$ java ThreadTest
    From ThreadA: i= 1
    From ThreadB: j= 1
    From ThreadA: i= 2
    From ThreadC: k= 1
    From ThreadB: j= 2
    From ThreadA: i= 3
    From ThreadC: k= 2
    From ThreadB: j= 3
    From ThreadA: i= 4
    From ThreadC: k= 3
    From ThreadB: j= 4
    From ThreadA: i= 5
    From ThreadC: k= 4
    From ThreadB: j= 5
Exit from A
    From ThreadC: k= 5
Exit from B
Exit from C
```


POSIX Thread Programming

- Standard Thread Library for POSIX-compliant systems
- Supports thread creation and management
- PThreads have type `pthread_t`.
- There are 5 basic pthread functions: (Tab 13.1)
 - `pthread_create`: creates a new thread within a process.
 - `pthread_join`: waits for another thread to terminate.
 - `pthread_equal`: compares thread ids to see if they refer to the same thread.
 - `pthread_self`: returns the id of the calling thread.
 - `pthread_exit`: terminates the currently running thread.

Pthread vs. Java Thread vs. OpenMP

```
#include <pthread.h>
#include <stdio.h>
```

```
void* thrfunc(void* arg)
```

```
{printf("h
```

```
int main(
```

```
{
```

```
pthread
```

```
pthread
```

```
int arg
```

```
int i;
```

```
// setu
```

```
pthread
```

```
pthread
```

```
pthread
```

```
// crea
```

```
for(i=0
```

```
pthread
```

```
// wait
```

```
for(i=0
```

```
pthread
```

```
class myth extends Thread
```

```
{
```

```
    myth (String name)
```

```
    {super(name); }
```

```
public
```

```
{
```

```
    #include <omp.h>
```

```
    #include <stdio.h>
```

```
    #define NUM_THREADS 4
```

```
}
```

```
int main(void)
```

```
{
```

```
    omp_set_num_threads(NUM_THREADS);
```

```
class He
```

```
{
```

```
    #pragma omp parallel
```

```
    printf("Hello from thread %d\n", omp_get_thread_num());
```

```
{
```

```
    for(int i=0; i<NUM_THREADS; i++)
```

```
}
```

```
        new myth("Thread " + i).start();
```

```
}
```

```
}
```



HPC与并行计算：多线程并发举例

• 原理

- 利用多线程编程可在多个处理器核上并行执行计算任务；
- 简单分发独立任务时若每个核分得的任务过小时，线程管理、任务分发和同步等操作代价比例增大可能导致最终性能下降。

• 鲲鹏特点

- 鲲鹏920处理器包含48核或64核和2个NUMA节点（24或32核属于一个节点）。

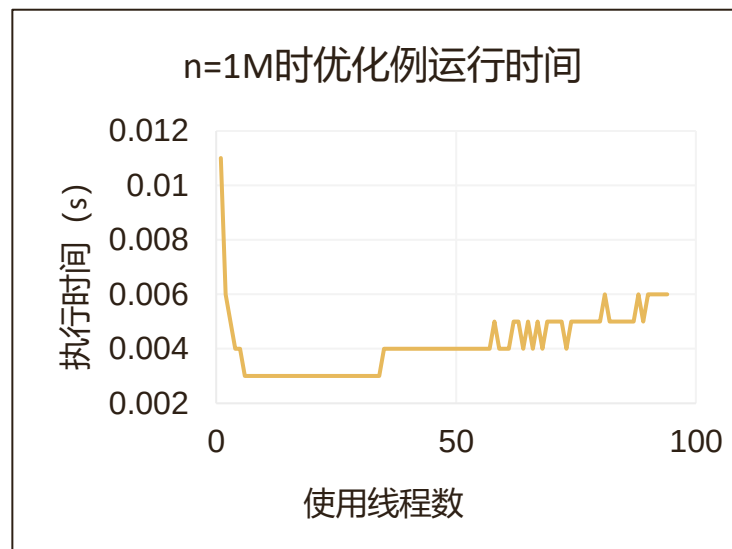
• 优化方案

- 任务粒度过细管理同步代价比例增大导致性能下降，建议通过实测确定每个核的最小任务大小（即最佳线程数）。

• 优化例（见右侧）

- 在for语句前增加编译指导，循环会被拆分并行执行；
- 调整OMP_NUM_THREADS环境变量发现N=1M时最优线程数为8-34。

```
void VecExp(long n, const float *a, float *y)
{
    #pragma omp parallel for
    for (long i = 0; i < n; i++) {
        y[i] = expf(a[i]);
    }
}
```



共享存储编程

多线程

OpenMP Classic

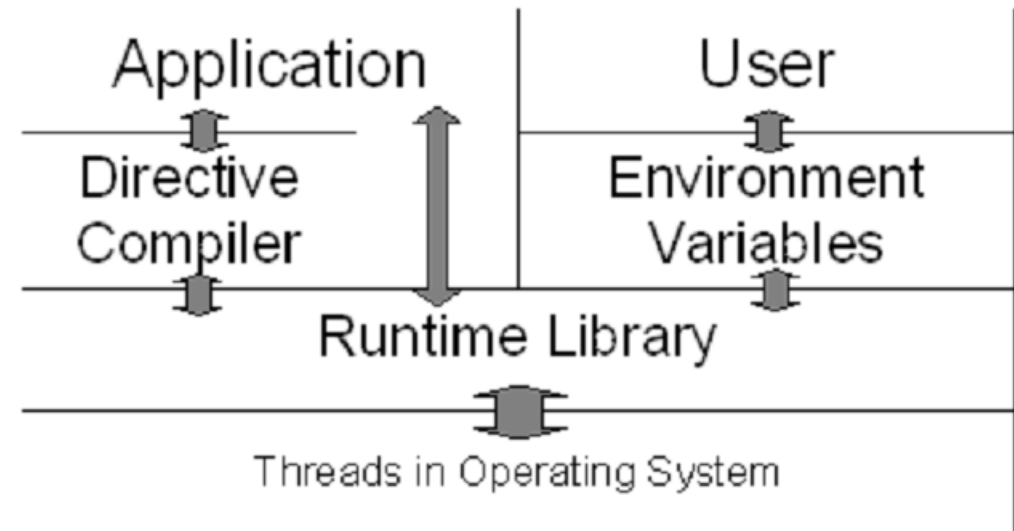
OpenMP Offload

What is OpenMP ?

- OpenMP (Open specifications for multi-processing), an Application Program Interface (API) to be used to explicitly direct **multi-threaded, shared memory parallelism**
- History
 - 1997: Version 1.0 for Fortran
 - 1998: Version 1.0 for C/C++
 - 2002-2005: Versions 2.0-2.5, Merger of Fortran and C/C++ specifications
 - 2008: Version 3.0, Incorporates Task Parallelism
 - 2013: Version 4.0, **Support for Accelerators**, SIMD support
 - 2018: Version 5.0, C11/C++17/Fortran 2008 support

OpenMP Architecture

- Three API components
 - Compiler Directives (编译制导)
 - Runtime Library Routines (运行库例程)
 - Environment Variables (环境变量)



Features

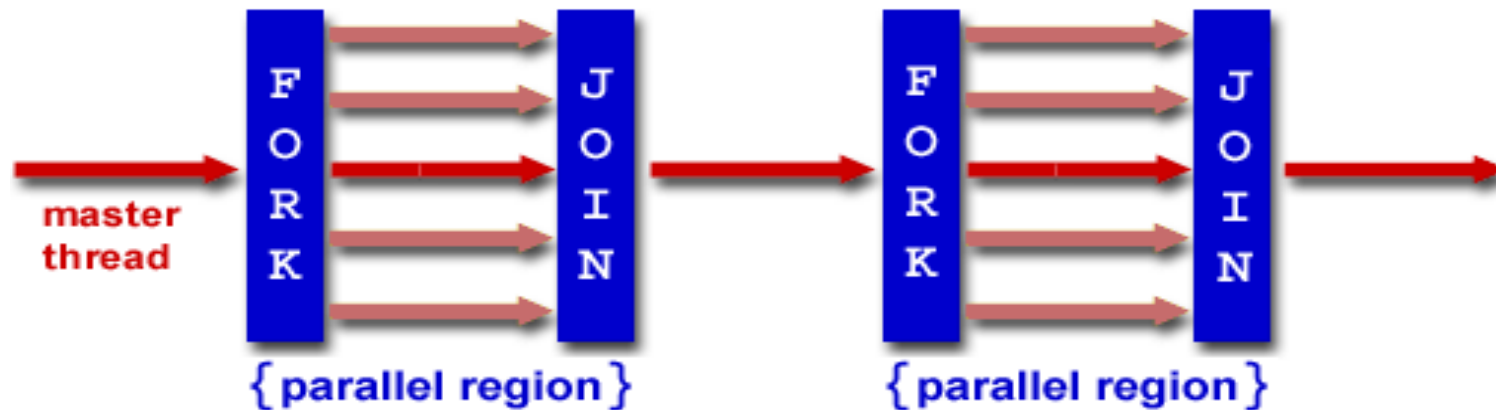
- Standardization
 - Provide a standard among a variety of shared memory architectures (platforms)
 - High-level interfaces to thread programming
- Lean and Mean
 - A simple and limited set of directives for shared address space programming
 - Just 3 or 4 directives are enough to represent significant parallelism
- Ease of use
 - Incrementally parallelize (增量并行化) a serial program
 - Unlike all or nothing approach of message-passing
 - Implement both coarse-grain and fine-grain parallelism
- Portability
 - Fortran (77, 90, and 95), C, and C++
 - Public forum for API and membership

Programming Model

- **Thread Based Parallelism**
 - A shared memory process with multiple threads
 - Explicit Parallelism: offer the programmer full control over parallelization
- **Compiler Directive (编译制导) Based**
 - Parallelism is specified through the use of compiler directives imbedded in C/C++ or Fortran source code
- **Nested Parallelism Support**
 - Parallel constructs may include other parallel constructs inside
 - Implementation-dependent
- **Dynamic Threads**
 - Alter the number of threads used to execute parallel regions
 - Implementation-dependent

Fork-Join Model

- All OpenMP programs begin as a single sequential process: the **master thread**
- **Fork** at the beginning of parallel constructs
 - The master thread creates a **team** of parallel threads
 - The statements enclosed by the parallel region construct are executed in parallel
- **Join** at the end of parallel constructs
 - The threads synchronize and terminate after completing the statements in the parallel construct
 - Only the master thread exists



General Code Structure

```
#include <omp.h>
void saxpy()
{
    float a, x[ARRAY_SZ], y[ARRAY_SZ];
    /* Beginning of parallel section. Fork a team of threads.
       Specify variable scoping */
    #pragma omp parallel
    {
        int id=omp_get_thread_num();
        int nthrs=omp_get_num_threads();
        for (int i=id; i < ARRAY_SZ; i+=nthrs) {
            y[i] = a * x[i] + y[i];
        }
    }
}
```

Terms

- Construct (结构)
 - A statement, which consists of a directive and the subsequent structured block.
- Directive (制导)
 - A C or C++ #pragma followed by the omp identifier, other text, and a new line.
 - The directive specifies program behavior.
- Structured block (结构块)
 - A structured block is a statement that has a single entry and a single exit.
 - Enclosed in {.....}
- Master thread (主线程)
 - The thread that creates a team when a parallel region is entered.
- Team (线程组)
 - One or more threads cooperating in the execution of a construct.

Terms (Cont.)

- Parallel region (并行区域)
 - Statements that bind to an OpenMP parallel construct and may be executed by multiple threads.
- Serial region (串行区域)
 - Statements executed only by the master thread outside of the dynamic extent of any parallel region.
- Private (私有的)
 - A private variable names a block of storage that is unique to the thread making the reference.
- Shared (共享的)
 - A shared variable names a single block of storage.
 - All threads in a team that access this variable will access this single block of storage.

OpenMP Classic

Parallel region (并行域)

Work-Sharing Constructs (共享任务结构)

- #pragma omp for
- #pragma omp single
- #pragma omp sections

Combined Parallel Work-Sharing
Constructs (组合的共享任务结构)

- #pragma omp parallel for
- #pragma omp parallel sections

Synchronization Constructs (同步结构)

Data environment (数据环境)

- #pragma omp threadprivate

Data Scope Clauses (数据域属性子句)

Directive Format

#pragma omp	Directive name	[clause, ...]	newline
Start of OpenMP C/C++ directives	Valid OpenMP directive, After the pragma and before any clauses	In any order Can be repeated	Required, Proceeds the structured block enclosed by this directive

Example: #pragma omp parallel default(shared) private(beta, pi)

Parallel Region

- A block of code to be executed by multiple threads.
- The fundamental OpenMP parallel construct
- Format
 - #pragma omp parallel [clause ...] newline
 - if (scalar_expression)
 - private (list)
 - shared (list)
 - default (shared | none)
 - firstprivate (list)
 - reduction (operator: list)
 - copyin (list)
 - structured_block

Parallel Directive (Cont.)

- In reaching a **PARALLEL** directive, a thread creates a team of threads and becomes the master. The master is a member of that team. The code is duplicated and all threads will execute that code.
- The # of threads of team
 - Determined by the following factors, in order of precedence:
 - `omp_set_num_threads()` library function
 - `OMP_NUM_THREADS` environment variable
 - Threads are numbered from 0 (**master thread**) to N-1
- Dynamic Threads
 - By default, a program uses the same number of threads to execute each parallel region.
 - Can dynamically adjust the number of threads
 - `omp_set_dynamic()` library function
 - `OMP_DYNAMIC` environment variable
- An implied barrier at the end of a parallel section. Only the master thread continues execution past this point.

Example of **Parallel** Region: Calculate PI

```
#include <omp.h>
static long num_steps = 100; double step;
#define NUM_THREADS 16

void main ()
{ int i; double x, pi, sum[NUM_THREADS];
  step = 1.0/(double) num_steps;
  omp_set_num_threads(NUM_THREADS);

  #pragma omp parallel
  { double x; int id; int i;
    id = omp_get_thread_num();

    printf("this is thread %d.\n",id);
    for (i=id, sum[id]=0.0;i< num_steps; i=i+NUM_THREADS){
      x = (i+0.5)*step;
      sum[id] += 4.0/(1.0+x*x);
    }
  }
  for(i=0, pi=0.0;i<NUM_THREADS;i++) pi += sum[i] * step;
  printf("Program successfully terminated!\n");
  printf("PI is %lf.\n",pi);
}
```

```
this is thread 11.
this is thread 14.
this is thread 8.
this is thread 12.
this is thread 7.
this is thread 6.
this is thread 13.
this is thread 10.
this is thread 5.
this is thread 4.
this is thread 3.
this is thread 15.
this is thread 9.
this is thread 1.
this is thread 2.
this is thread 0.
Program successfully terminated!
PI is 3.141601.
```



Work-Sharing Constructs

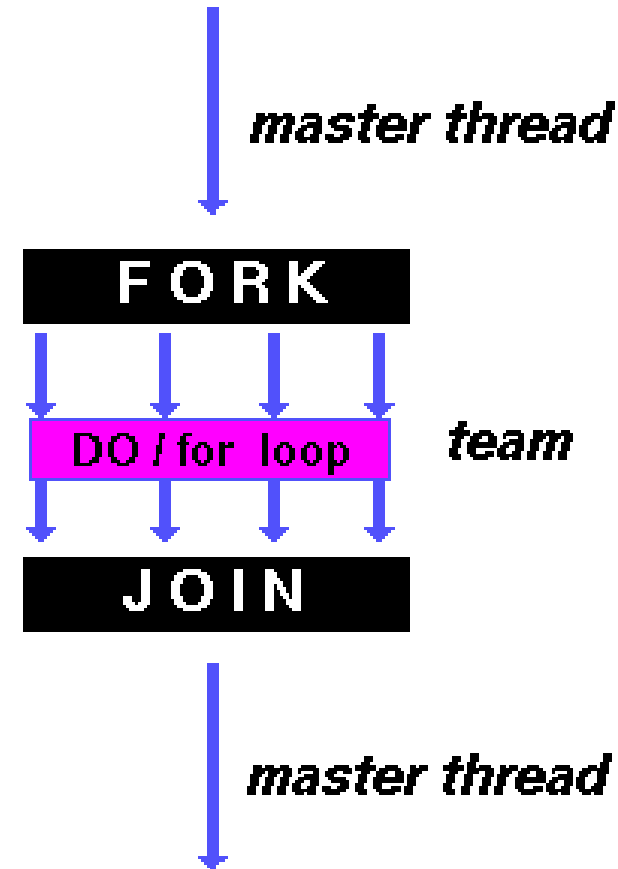
- Divides the execution of the region among the members of the team
- An implied barrier at the end of the constructs
- No implied barrier upon the entry of the constructs
- Work-sharing constructs do not launch new threads

Construct Types

- #pragma omp for
 - Shares iterations of a loop across the team.
 - Represents a type of **data parallelism**
- #pragma omp single
 - **Serializes** a section of code
- #pragma omp sections
 - Breaks work into separate, discreet sections.
 - Each section is executed by a thread.
 - Can be used to implement a type of **functional parallelism**

#pragma omp for

- The iterations of the loop immediately following this directive must be executed in parallel by the team
- This assumes a parallel region has already been initiated
 - Otherwise it executes in serial on a single processor



#pragma omp for (Cont.)

- Format

#pragma omp for [clause ...] newline

schedule (type [,chunk])

ordered

private (list)

firstprivate (list)

lastprivate (list)

shared (list)

reduction (operator: list)

nowait

for_loop

Type & Chunk in Schedule Clause

- **static**: Iterations are divided into pieces of a size specified by **chunk**. The pieces are statically assigned to threads in a round robin fashion.
- **dynamic**: Iterations are broken into pieces of a size specified by **chunk**. As each thread finishes a piece, it dynamically obtains the next set of iterations.
- **guided**: The chunk size is reduced in an exponentially decreasing manner. **chunk** specifies the smallest piece.
- **runtime**: The decision regarding scheduling is deferred until run time.
- The schedule type and chunk size can be chosen at run time by setting the OMP_SCHEDULE environment variable.

Loop scheduling

static



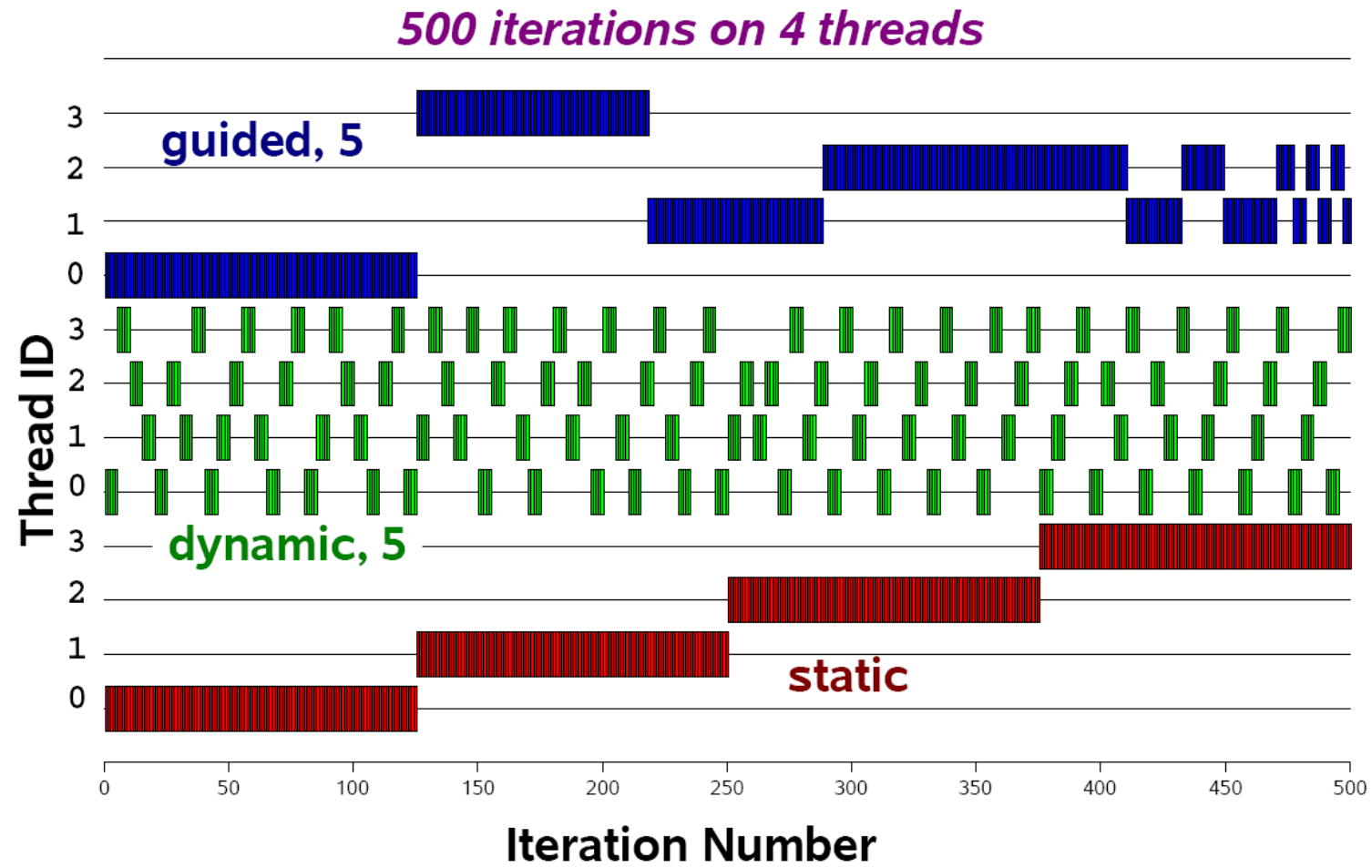
dynamic(3)



guided(1)



Example: 500 iterations on 4 threads



Example of For Directive

```
#include <omp.h>
#define CHUNKSIZE 100
#define N    1000

main ()
{
    int i, chunk;
    float a[N], b[N], c[N];

    /* Some initializations */
    for (i=0; i < N; i++)
        a[i] = b[i] = i * 1.0;
    chunk = CHUNKSIZE;

    #pragma omp parallel shared(a,b,c,chunk) private(i)
    {
        #pragma omp for schedule(dynamic,chunk) nowait
        for (i=0; i < N; i++)
            c[i] = a[i] + b[i];
    } /* end of parallel section */
}
```



Example of Work-Sharing Constructs : Calculate PI

```
#include <omp.h>
static long num_steps = 100; double step;
#define NUM_THREADS 16
void main ()
{
    int i; double x, pi, sum[NUM_THREADS];
    step = 1.0/(double) num_steps;
    omp_set_num_threads(NUM_THREADS);

    #pragma omp parallel
    { double x; int id;
      id = omp_get_thread_num(); sum[id] = 0;
      printf("this is thread %d.\n",id);

      #pragma omp for
      for (i=0;i< num_steps; i++){
          x = (i+0.5)*step;
          sum[id] += 4.0/(1.0+x*x);
      }
    }
    for(i=0, pi=0.0;i<NUM_THREADS;i++)pi += sum[i] * step;
    printf("Program successfully terminated!\n");
    printf("PI is %lf.\n",pi);
}
```

```
this is thread 7.
this is thread 9.
this is thread 6.
this is thread 2.
this is thread 11.
this is thread 13.
this is thread 3.
this is thread 15.
this is thread 5.
this is thread 12.
this is thread 14.
this is thread 4.
this is thread 8.
this is thread 1.
this is thread 0.
this is thread 10.
Program successfully terminated!
PI is 3.141601.
```

Combined Parallel Work-Sharing Constructs

- `#pragma omp parallel for`
 - Simplified form of `#pragma omp parallel` + `#pragma omp for`
- `#pragma omp parallel sections`
 - Simplified form of `#pragma omp parallel` + `#pragma omp sections`

#pragma omp parallel for

```
#include <omp.h>
#define N    1000
#define CHUNKSIZE  100

main () {
    int i, chunk;
    float a[N], b[N], c[N];

    /* Some initializations */
    for (i=0; i < N; i++)
        a[i] = b[i] = i * 1.0;
    chunk = CHUNKSIZE;

    #pragma omp parallel for shared(a,b,c,chunk) private(i) schedule(static,chunk)
    for (i=0; i < n; i++)
        c[i] = a[i] + b[i];
}
```



OpenMP Synchronization

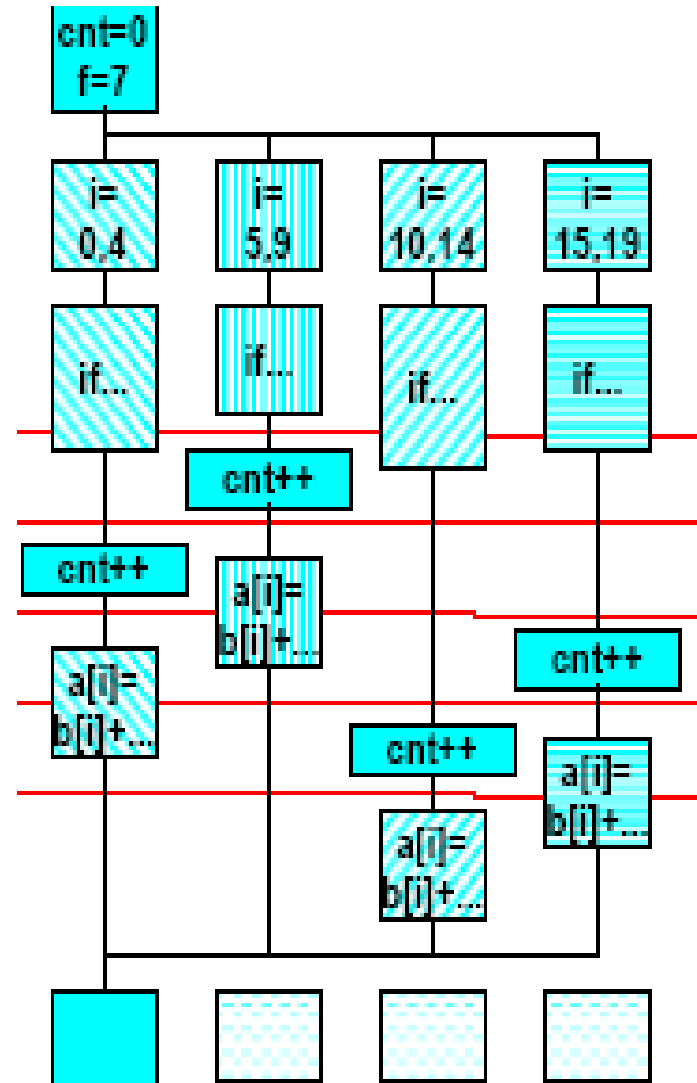
- Implicit Barrier
 - beginning and end of parallel constructs
 - end of all other control constructs
 - implicit synchronization can be removed with nowait clause
- Explicit: Synchronization Constructs
 - #pragma omp critical
 - #pragma omp master
 - #pragma omp barrier
 - #pragma omp atomic
 - #pragma omp flush
 - #pragma omp ordered

#pragma omp critical

- Specifies a region of code that must be executed by only one thread at a time.
- Race condition
 - Other thread will block until the first thread exits the CRITICAL region
- Format
 - #pragma omp critical [name]
 newline
 structured_block

Example of Critical Directive

```
cnt = 0;  
f=7;  
#pragma omp parallel  
{  
#pragma omp for  
  for (i=0; i<20; i++) {  
    if (b[i] == 0) {  
      #pragma omp critical  
      cnt ++;  
    } /* endif */  
    a[i] = b[i] + f * (i+1);  
  } /* end for */  
} /*omp end parallel */
```



Summary: Control Structures

- Parallel region construct
 - parallel
- Work-sharing constructs
 - sections, for, single
- Combined parallel work-sharing construct
 - parallel for, parallel section
- Synchronization constructs
 - critical, master, barrier, atomic, flush, ordered

Data Scope Clauses

- Data scope attribute clauses
 - Explicitly define how variables should be scoped
 - An important consideration for OpenMP programming is the understanding and use of data scoping
 - Most variables are shared by default
- Global variables include
 - File scope variables, static
- Private variables include
 - Loop index variables
 - Stack variables in subroutines called from parallel regions

Kinds of Data Scope Clauses

- #pragma ... shared
- #pragma ... private
- #pragma ... firstprivate
- #pragma ... lastprivate
- #pragma ... threadprivate

- #pragma ... default
- #pragma ... reduction
- #pragma ... copyin

Private and Shared Clause

- Private Clause

- Declares variables in its list to be private to each thread
- Format
 - private (list)

- Shared Clause

- Declares variables in its list to be shared among all threads in the team
- Format
 - shared (list)

Default Clause

- Allows the user to specify a default SHARED, or NONE scope for all variables in the lexical extent of any parallel region.
- Format
 - default (shared | none)
- Notes
 - Specific variables can be exempted from the default using the PRIVATE, SHARED, FIRSTPRIVATE, LASTPRIVATE, and REDUCTION clauses

```
#pragma omp parallel default(none) \  
    shared(a,b,c) private(i)
```

Reduction Clause

- Performs a reduction on the variables that appear in its list.
- A private copy for each list variable is created for each thread.
- At the end of the reduction, the reduction variable is applied to all private copies of the shared variable, and the final result is written to the global shared variable
- Format
 - reduction (operator: list)

Reduction Example

```
#include <omp.h>

main () {
    int i, n, chunk;
    float a[100], b[100], result;

    /* Some initializations */
    n = 100;
    chunk = 10;
    result = 0.0;
    for (i=0; i < n; i++) {
        a[i] = i * 1.0;
        b[i] = i * 2.0;
    }

    #pragma omp parallel for default(shared) private(i) schedule(static,chunk) \
        reduction(+:result)
    for (i=0; i < n; i++)
        result = result + (a[i] * b[i]);
    printf("Final result= %f\n",result);
}
```

Final result= 656700.000000



Environment Variables

- OMP_SCHEDULE
 - Applies only to for, parallel for directives which have their schedule clause set to RUNTIME
 - setenv OMP_SCHEDULE "guided, 4"
 - setenv OMP_SCHEDULE "dynamic"
- OMP_NUM_THREADS
 - Sets the maximum number of threads to use during execution.
 - setenv OMP_NUM_THREADS 8
- OMP_DYNAMIC
 - Enables or disables dynamic adjustment of the number of threads available for execution of parallel regions.
 - setenv OMP_DYNAMIC TRUE
- OMP_NESTED
 - Enables or disables nested parallelism.
 - setenv OMP_NESTED TRUE

OpenMP Compiler Support

- GCC/G++
 - gcc -fopenmp <source>.c
 - g++ -fopenmp <source>.cpp

Summary

- Java thread
 - Threading Mechanisms
 - Thread cycle
 - Synchronization
 - Priority
- OpenMP
 - Program model: fork-join model
 - Components: Directives, Runtime libraries, Environment variables
 - Parallel region
 - Work-Sharing Constructs
 - Synchronization Constructs
 - Data Scope Clauses